CNS Practical Crypto Exam Handbook

Simple English – practical attacks, code, and variants

28 October 2026 | Prepared for the 29 October exam

Key idea

This is a practical open-note guide for three CTF-style exercises. It does not contain theory quizzes. For every attack, learn: (1) what clue identifies it, (2) why it works, (3) what each important Python line does, (4) what to change in a new problem, and (5) how to check your answer. Tested challenge flags are marked as confirmed; other examples are learning templates, not confirmed solutions.

Contents

1 One-page attack finder 2

2 Python essentials and crypto basics 2
- 2.1 Bytes, hex, text, ASCII, base64url . . . 2
- 2.2 XOR: the operation behind many attacks . . . 2
- 2.3 Hash, HMAC, encryption, encoding and nonce . . . 2

3 Basic cipher toolkit – try these before hard attacks 3
- 3.1 Caesar cipher: fixed alphabet shift . . . 3
- 3.2 Fixed substitution alphabet . . . 3
- 3.3 Vigenere: repeated keyword . . . 3
- 3.4 Belgian message: word-list substitution workflow . . . 4
- 3.5 Dutch hex ciphertext and hex key: XOR toolkit . . . 4

4 First Enigma Machine – rotating substitutions 5

5 Replanting – stream reuse and a weak random generator 6

6 Noncense – AES-CTR nonce reuse 7

7 Key standardisation – DES with a weak key pattern 7

8 AES2 – meet-in-the-middle (MITM) 8

9 Better Hash – reverse a home-made function 8

10 SecureBank 1 – Salsa20 bit-flipping 9

11 SecureBank 2 – truncated SHA-256 collision 9

12 Mars Relay – hash chain prediction 10

13 Signed, Sealed, Misinterpreted – HMAC and duplicate JSON keys 10

14 Vault JWT – unsigned `alg: none` token 11

15 Updated practical solutions: use these versions first 11
- 15.1 AES2: discover ECB alignment, infer BMP masks, solve MITM . . . 11
- 15.2 Better Hash: invert every step in reverse order . . . 12
- 15.3 Enigma: stepping and inverse permutations . . . 13
- 15.4 Replanting: stream XOR + flexible LCG state search . . . 13
- 15.5 Quick practical fallback: when you cannot complete a challenge . . . 14

16 Universal exam workflow and error checklist 14
- 16.1 How to solve a changed question without AI . . . 14
- 16.2 Common Python problems . . . 15

1 One-page attack finder

Clue in the question	Best first idea	Core Python technique
Alphabet shifted by a fixed amount	Caesar / ROT	Try 26 shifts, preserve punctuation
Two alphabets, one plain and one cipher	Monoalphabetic substitution	Reverse the mapping
A keyword or repeated alphabet shifts	Vigenere	Subtract key-letter shifts
English word list supplied	Substitution / dictionary search	Word patterns and frequency analysis
Long hex message and hex key	XOR (repeating key possible)	`bytes.fromhex`, modulo key length
Three rotating alphabets	Enigma-like rotor cipher	Rotate and invert rotors in reverse order
Known plaintext + another stream ciphertext	Keystream reuse	$P_2 = C_1 \oplus P_1 \oplus C_2$
Linear generator in code	LCG state recovery	Recover state from known output bytes
DES key based on a short name	Weak key derivation	Generate and test candidate keys
Two AES keys of only 20 bits each	Meet-in-the-middle	Match $E_{K_1}(P)$ with $D_{K_2}(C)$
Home-made hash using XOR and rotation	Reverse all rounds	Undo in reverse order, use ROR
Salsa20 and editable payment	Bit-flipping	$C' = C \oplus P \oplus P'$
Short SHA-256 fingerprint	Birthday collision	Dictionary of seen fingerprints
Next code based on previous hash	Hash chain prediction	Hash raw bytes from hex
HMAC-protected JSON, duplicate fields	Parsing/canonicalisation mismatch	Compare verification and authorisation
JWT with algorithm in header	`alg: none` acceptance	Create unsigned JWT for lab

2 Python essentials and crypto basics

2.1 Bytes, hex, text, ASCII, base64url

A byte contains 8 bits. Two hex digits represent one byte. A hex string is a way to display data, not encryption. Python `bytes` objects are immutable; `bytearray` objects can be changed. Slices use byte positions when the object contains bytes, and the final index is excluded.

```
raw = bytes.fromhex('3130')       # b'10', two bytes
raw.hex()                         # '3130'
len(raw)                          # 2 (bytes, not bits)
raw[:1], raw[1:], raw[-1:]        # first, after first, last
b'Hello'.decode('utf-8')          # bytes -> string
'Hello'.encode('utf-8')           # string -> bytes
ord('1'), ord('0')                # 49, 48
mutable = bytearray(raw)          # now mutable
mutable[0] ^= 1                   # flip lowest bit

import base64
s = base64.urlsafe_b64encode(b'hello').decode().rstrip('=')
b = base64.urlsafe_b64decode(s + '=' * (-len(s) % 4))
```

2.2 XOR: the operation behind many attacks

XOR gives 1 when two bits are different, and 0 when they are equal. Important identities are $a \oplus a = 0$, $a \oplus 0 = a$, and $(a \oplus b) \oplus b = a$. A stream cipher encrypts with $C = P \oplus K$ and decrypts with $P = C \oplus K$. Here K is the keystream, not necessarily the raw secret key.

```
def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

# Only the shortest input length is processed by zip.
assert len(b'ABC') == len(b'xyz')
print(xor_bytes(b'ABC', b'xyz').hex())
```

2.3 Hash, HMAC, encryption, encoding and nonce

A **hash** maps data to a fixed-size output; SHA-256 gives 256 bits. **HMAC** uses a secret key to authenticate data. **Encryption** hides the message; a stream cipher normally needs a nonce and key. **Base64 and hex** only encode

data and do not hide or authenticate it. A **nonce** is normally public and must not be reused with the same key in stream-cipher constructions that require uniqueness.

```
import hashlib, hmac, json
hash_hex = hashlib.sha256(b'example').hexdigest()
tag = hmac.new(b'secret', b'example', hashlib.sha256).hexdigest()
canonical = json.dumps({'b': 2, 'a': 1},
                       sort_keys=True, separators=(',', ':'),
                       ensure_ascii=True).encode()
# b'{"a":1,"b":2}'
```

3 Basic cipher toolkit – try these before hard attacks

3.1 Caesar cipher: fixed alphabet shift

Question: "My name is Caesar" with `fqv{iluvw-fkdoohqjh}`. A Caesar cipher shifts each letter by the same number k. To decrypt, subtract k modulo 26. The simplest universal strategy is to print all 26 options and look for normal words or a flag-like structure. The commonly used Caesar shift of 3 gives `cns{first-challenge}` for this example.

```
import string
ABC = string.ascii_lowercase
ciphertext = 'fqv{iluvw-fkdoohqjh}'

def caesar(text, shift):
    out = ''
    for ch in text:
        if ch.lower() in ABC:
            base = ord('A') if ch.isupper() else ord('a')
            out += chr((ord(ch) - base - shift) % 26 + base)
        else:
            out += ch
    return out

for shift in range(26):
    print(shift, caesar(ciphertext, shift))
```

Adapt: change `ciphertext`. If encryption is requested, add the shift rather than subtract it. Keep case, braces, hyphens and spaces unchanged. Check whether the task uses 0–25 for A–Z.

3.2 Fixed substitution alphabet

Question: Plain ABCDEFGHIJKLMNOPQRSTUVWXYZ; Cipher QWERTYUIOPASDFGHJKLZXCVBNM; encrypted text `EFL{STZZTK-LGXH}`. In a substitution cipher each letter maps to one other letter. This is not a Caesar shift, because the mapping is not a uniform movement. For decryption, find each cipher letter's position in the cipher alphabet and take that position from the plain alphabet.

```
PLAIN  = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
CIPHER = 'QWERTYUIOPASDFGHJKLZXCVBNM'
text = 'EFL{STZZTK-LGXH}'
assert len(PLAIN) == len(CIPHER) == 26
assert len(set(CIPHER)) == 26  # must be a permutation

def substitute_decrypt(text, plain=PLAIN, cipher=CIPHER):
    return ''.join(plain[cipher.index(c)] if c in cipher else c
                   for c in text)
print(substitute_decrypt(text))  # CNS{LETTER-SOUP}
```

Adapt: replace `CIPHER` with the provided scrambled alphabet. To encrypt, use `cipher[plain.index(ch)]`. If the alphabet is lowercase, adjust case or use `text.upper()`.

3.3 Vigenere: repeated keyword

French question: `eeg{kjgiflceha-sje-oxvksz-lzaa-ikqni}`. The hint is a French moon-shaped breakfast item: `croissant`. This suggests Vigenere encryption, where each letter is shifted by a letter of the repeating key. For A=0, decryption uses $p_i = (c_i - k_i) \bmod 26$. Usually nonletters do not advance the key; if results look wrong, try the other convention.

```
ct = 'eeg{kjgiflceha-sje-oxvksz-lzaa-ikqni}'
key = 'croissant'
```

```python
def vigenere(text, key, decrypt=True, advance_on_symbols=False):
    result, pos = [], 0
    for ch in text:
        if ch.isascii() and ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            shift = ord(key[pos % len(key)].lower()) - ord('a')
            if decrypt: shift = -shift
            result.append(chr((ord(ch) - base + shift) % 26 + base))
            pos += 1
        else:
            result.append(ch)
            if advance_on_symbols: pos += 1
    return ''.join(result)

print(vigenere(ct, key))  # cns{croissants-are-better-than-pizza}
# If output makes no sense, test advance_on_symbols=True.
```

Adapt: change the keyword and ciphertext. If the hint is a word but not a key, test candidate words. If you see one shift for every position, this is different from monoalphabetic substitution. Check whether the key advances across braces and hyphens.

3.4 Belgian message: word-list substitution workflow

Question: `qnb{vsi-fihczlnb-dlyi-vsi-fibv-plaahib-fi-zv-l-hzici-plaahi-mr-l-frtbbihb-plaahi}`. The text says an English word list may help. The supplied prompt alone does *not* prove which cipher was used. A good first test is monoalphabetic substitution with English word patterns; if a separate attached word list or source code names another method, follow that method. Never claim a recovered flag unless it was checked.

For a substitution, repeated letters preserve a pattern. Example: `letter` has pattern 0-1-2-2-1-3. Find dictionary words with the same pattern, then test a consistent one-to-one mapping across all words. The solver below uses a small word-list filter; a full solution must also enforce consistent mapping between words.

```python
from collections import defaultdict

def pattern(word):
    ids = {}
    return tuple(ids.setdefault(ch, len(ids)) for ch in word)

cipher_word = 'plaahib'   # change for each word
with open('english_words.txt', encoding='utf-8') as f:
    words = [w.strip().lower() for w in f if w.strip().isalpha()]
candidates = [w for w in words if len(w) == len(cipher_word)
              and pattern(w) == pattern(cipher_word)]
print(candidates[:100])
# Next: verify ONE cipher letter maps to exactly ONE plain letter,
# and vice versa, across the whole message.
```

Adapt: replace `english_words.txt` by the attached word list path and replace `cipher_word`. Do not brute-force all 26! permutations. Use word shapes, repeated letters, frequency, and consistent cross-word mappings. If the full word list is missing, a final result is not verified.

3.5 Dutch hex ciphertext and hex key: XOR toolkit

Question: a long hex ciphertext and a hex key starting `636f6d707574...`. This key begins with ASCII text `computer...`. Test if the key is used once, repeated, or is a hint to another operation. **Important:** the supplied ciphertext ends with an isolated `1` and includes a space; that is not a well-formed even-length hex string. Verify it against the original exercise before solving.

```python
from itertools import cycle
cipher_hex = 'PASTE_CLEAN_EVEN_LENGTH_HEX_HERE'
key_hex = ('636f6d7075746572656e6e65747765726b736563757269'
           '747969737468656265737 4636f75727365')

def decode_hex_xor(cipher_hex, key_hex, repeat=True):
    c = bytes.fromhex(''.join(cipher_hex.split()))
    k = bytes.fromhex(''.join(key_hex.split()))
    if not k: raise ValueError('Empty key')
    if not repeat and len(c) != len(k):
        raise ValueError('Non-repeating XOR needs equal lengths')
    stream = (k[i % len(k)] for i in range(len(c))) if repeat else iter(k)
    return bytes(a ^ b for a, b in zip(c, stream))

# Example usage after replacing the placeholder above:
# plain = decode_hex_xor(cipher_hex, key_hex, repeat=True)
# print(plain.decode('utf-8', errors='replace'))
```

Adapt: inspect `len(ciphertext)`, `len(key)` and the decoded key string. A repeated key uses `k[i % len(k)]`; if key and ciphertext lengths match exactly, ordinary bytewise XOR may be enough. Do not add or delete a hex digit blindly to fix malformed input.

4 First Enigma Machine – rotating substitutions

Clues: three rotor alphabets, each an alphabet permutation, stepping before each letter. Rotor 1 steps every letter; rotor 2 after 26; rotor 3 after 676. You undo encryption by applying the inverse of rotor 3, then 2, then 1. The historical Enigma also had other parts; this teaching cipher is simpler.

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

rotor1 = "MACHINEBDFGJKLOPQRSTUVWXYZ"
rotor2 = "ORBITACDEFGHJKLMNPQSUVWXYZ"
rotor3 = "CIPHERABDFGJKLMNOQSTUVWXYZ"

ciphertext = "HKV{BSBU-B-SJYZNNB-DNFW-IXH-YTA-RPL-RTXOLWFQ}"

plaintext = ""

# Count how many letters have been processed
count = 0

# Process each character of the ciphertext
for char in ciphertext:

    # Only decrypt letters A-Z
    if char in alphabet:
        # Increase the counter by 1
        count += 1
        # Rotor 1 rotates before every letter
        # Move the first character to the end
        rotor1 = rotor1[1:] + rotor1[0]
        # Rotor 2 rotates after 26 letters
        if count % 26 == 0:
            rotor2 = rotor2[1:] + rotor2[0]
        # Rotor 3 rotates after 26*26 letters
        if count % (26 * 26) == 0:
            rotor3 = rotor3[1:] + rotor3[0]
        # DECRYPTION: reverse the rotor order
        # Reverse Rotor 3
        position3 = rotor3.index(char)
        letter3 = alphabet[position3]
        # Reverse Rotor 2
        position2 = rotor2.index(letter3)
        letter2 = alphabet[position2]
        # Reverse Rotor 1
        position1 = rotor1.index(letter2)
        original_letter = alphabet[position1]
        # Add the decrypted letter to the plaintext
        plaintext += original_letter

    else:
        # Keep symbols such as { } and - unchanged
        # Symbols do not rotate the rotors
        plaintext += char

# Display the decrypted message
print("Decrypted message:", plaintext)
```

Code explanation: `count` tracks only A–Z characters. `rotor1[1:] + rotor1[0]` rotates left by one. `if count % 26 == 0` means every 26 processed letters. `rotor.index(char)` gives the position where the output letter occurs; `alphabet[position]` reverses the rotor mapping.

Adapt checklist: replace all three rotor strings; confirm each contains 26 unique letters; change the starting offsets if the statement gives nonzero positions (rotate first by the given amount); check *when* stepping occurs, before or after processing; decide whether punctuation advances the rotors; confirm rotor order. Test by encrypting the recovered plaintext back with the same stepping rules.

5 Replanting – stream reuse and a weak random generator

Clue: you know the plaintext of one ciphertext, and both ciphertexts use the same stream cipher settings. If both used exactly the same keystream, then

$$C_1 = P_1 \oplus K, \qquad C_2 = P_2 \oplus K, \qquad P_2 = C_1 \oplus P_1 \oplus C_2.$$

This only recovers positions covered by the known plaintext. The ZIP also has a weak LCG-based keystream generator, so we can attempt to recover more by finding its internal state.

```
"""
Encryption module.

This module implements a custom stream cipher.
"""

# LCG parameters
A = 1103515245
C = 12345
M = 2**31


def next_state(state):
    """Generate the next PRNG state."""
    return (A * state + C) % M


def generate_keystream(seed, length):
    """Generate a pseudorandom keystream."""
    state = seed
    keystream = bytearray()

    for _ in range(length):
        state = next_state(state)
        keystream.append((state >> 16) & 0xff)

    return bytes(keystream)


def encrypt(plaintext, seed):
    """Encrypt plaintext using XOR."""
    keystream = generate_keystream(seed, len(plaintext))
    return bytes(
        p ^ k for p, k in zip(plaintext, keystream)
    )


def decrypt(ciphertext, seed):
    """Decrypt ciphertext using XOR."""
    return encrypt(ciphertext, seed)
```

Understand the LCG: $x_{n+1} = (Ax_n + C) \bmod M$. The output byte is `(state » 16) & 0xff`, so it reveals eight bits (bits 16–23) of the 31-bit state. The unknown parts of the first updated state are 7 high bits and 16 low bits: 2^{23} candidates. Several known output bytes filter wrong candidates.

```
# Files public.txt and secret.txt in the provided ZIP are ciphertexts.
from pathlib import Path
A, C, M = 1103515245, 12345, 2**31

def step(x): return (A * x + C) % M

def stream(seed, n):
    x, out = seed, bytearray()
    for _ in range(n):
        x = step(x)
        out.append((x >> 16) & 255)
    return bytes(out)

# Supply known plaintext and both ciphertexts in the right format.
c1 = bytes.fromhex(Path('public.txt').read_text().strip())
c2 = bytes.fromhex(Path('secret.txt').read_text().strip())
p1 = b'STATUS: SERVER ONLINE'  # MUST match actual known plaintext
known = bytes(a ^ b for a, b in zip(c1, p1))

first_state = None
for high in range(128):
    for low in range(65536):
```

```
        x1 = (high << 24) | (known[0] << 16) | low
        x = x1
        for expected in known[1:]:
            x = step(x)
            if (x >> 16) & 255 != expected:
                break
        else:
            first_state = x1
            break
    if first_state is not None: break
assert first_state is not None, 'Check plaintext and file formats'
seed = ((first_state - C) * pow(A, -1, M)) % M
plain2 = bytes(a ^ b for a, b in zip(c2, stream(seed, len(c2))))
print(plain2.decode(errors='replace'))
```

Adapt: if a new exercise uses a different PRNG, copy its A, C, M and output function exactly. A different output shift changes which bits are unknown. Verify whether the file is hex text or raw bytes. If no weak PRNG is present, keystream reuse only gives the portion where plaintext is known.

6 Noncense – AES-CTR nonce reuse

CTR mode turns AES into a keystream generator: it encrypts successive counter blocks and XORs them with plaintext. If the same key and initial counter/nonce are repeated, the same keystream is reused. This is the same XOR weakness as Replanting, even though AES is a block cipher underneath.

```
# Files from ZIP: ciphertext_known.hex, ciphertext_flag.hex,
# known_message.txt (put in the same working directory).
from pathlib import Path
ck = bytes.fromhex(Path('ciphertext_known.hex').read_text().strip())
cf = bytes.fromhex(Path('ciphertext_flag.hex').read_text().strip())
known = Path('known_message.txt').read_bytes()
assert len(ck) >= len(cf) and len(known) >= len(cf)
flag = bytes(a ^ b ^ c for a, b, c in zip(ck, known, cf))
print(flag.decode(errors='replace'))
```

Adapt: confirm the nonce *and* key are reused and the counter starts at the same point. Confirm messages line up at byte zero. If you only know a plaintext prefix, you only recover a prefix of the other plaintext. A good defence is to use authenticated encryption with correct nonce handling.

7 Key standardisation – DES with a weak key pattern

Clue: namespace `IOTDEV` and DES. A DES key contains 8 bytes. The original exercise script assumes the final two characters are decimal digits, creating 100 candidate keys such as `IOTDEV00` to `IOTDEV99`. This is a weak key-generation method, not a weakness in every possible DES key.

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import unpad

namespace = "IOTDEV"

ciphertext = "74eb40cf03b84d0bc4cd2c6e0fe97fb4e70546d3e09c530d35b344e530351bac"

ciphertext_bytes = bytes.fromhex(ciphertext)

# Try keys formed from IOTDEV + two digits
for number in range(100):

    # Create an 8-character key
    key = (namespace + f"{number:02d}").encode("utf-8")

    # Create DES cipher using ECB mode
    cipher = DES.new(key, DES.MODE_ECB)

    # Decrypt the ciphertext
    plaintext = cipher.decrypt(ciphertext_bytes)

    try:
        # Remove PKCS7 padding if present
        plaintext = unpad(plaintext, 8)

        # Convert bytes to readable text
        message = plaintext.decode("utf-8")
    except (ValueError, UnicodeDecodeError):
```

```
        # Incorrect key or invalid padding/text
        continue
    else:
        print("Possible key:", key)
        print("Message:", message)
        break
```

Code explanation: `f"{number:02d}"` creates two digits with a leading zero if needed. `DES.new(key, DES.MODE_ECB)` selects DES in ECB mode. `unpad(...,8)` removes PKCS#7 padding; invalid padding usually means a bad key. `decode('utf-8')` is an extra readability check, not proof.

Adapt: change the namespace, missing characters and key structure. If key generation is not known, do not assume two digits. DES uses an 8-byte key representation. Confirm the mode (ECB/CBC), IV, encoding and padding. CBC also requires an IV; the template above is ECB-only. Validate candidate plaintext format, not just successful padding.

8 AES2 – meet-in-the-middle (MITM)

Clue: encryption applies two AES operations using separate 20-bit keys. Each key is made from a 20-bit number encoded in three big-endian bytes, plus thirteen zero bytes. The idea that 20+20 bits always gives 40 bits of security is wrong for this construction.

$$C = E_{K_2}(E_{K_1}(P)), \quad E_{K_1}(P) = D_{K_2}(C).$$

Attack: store all possible $E_{K_1}(P)$, then test all $D_{K_2}(C)$ for a match. Complexity is about $2^{20} + 2^{20}$ AES operations and about 2^{20} stored values, rather than 2^{40} trials. Memory in Python can be much larger than the raw values due to dictionary overhead.

```
from Crypto.Cipher import AES

def make_key(n):
    return n.to_bytes(3, 'big') + b'\x00' * 13

def mitm(known_plain_block, known_cipher_block):
    assert len(known_plain_block) == len(known_cipher_block) == 16
    table = {}
    for k1 in range(2**20):
        mid = AES.new(make_key(k1), AES.MODE_ECB).encrypt(known_plain_block)
        table.setdefault(mid, []).append(k1)
    for k2 in range(2**20):
        mid = AES.new(make_key(k2), AES.MODE_ECB).decrypt(known_cipher_block)
        if mid in table:
            for k1 in table[mid]:
                yield k1, k2   # verify with another known block

# for k1, k2 in mitm(P, C):
#     if verify_on_second_block(k1, k2): print(k1, k2); break
```

Where do P and C come from? The provided project uses a BMP image and considers known 16-byte colour-mask blocks. The ciphertext block is extracted at offset 54, after the typical BMP header. That offset and plaintext assumption are *file-specific*. Your program must test plausible known plaintext blocks and verify the resulting keys against additional data. The promo ciphertext is then decrypted as $D_{K_1}(D_{K_2}(C))$. Do not assume you have a real match from only an unverified guess.

Adapt: replace key width (`2**20`), the `make_key` format, cipher mode, order of the two encryption steps, and the file offset. MITM needs at least one known plaintext/ciphertext pair for the double encryption. If the mode is CBC or CTR, account for IV/nonce and block dependence; do not apply the ECB example unchanged.

9 Better Hash – reverse a home-made function

Clue: the function changes every byte by XOR, rotates bits, mixes each byte with the preceding byte, then rotates the array. This is *not* a safe cryptographic hash because it keeps the message length and uses reversible operations. To invert it, undo each operation in reverse order, and undo the eight rounds in reverse order.

```
def rol(x, n): return ((x << n) | (x >> (8-n))) & 255
def ror(x, n): return ((x >> n) | (x << (8-n))) & 255

hash_hex = ('5f070b54361613bca31a123e95321cff154ba64f'
            '549bb34fdc7688f10efb569aed4ccbd6ba')
state = bytearray.fromhex(hash_hex)
for rnd in range(7, -1, -1):
    # Undo left rotation of entire byte array.
    state = state[-1:] + state[:-1]
```

```
    # Undo chained XOR from right to left.
    for i in range(len(state)-1, 0, -1):
        state[i] ^= state[i-1]
    # Undo each byte's bit rotation.
    for i in range(len(state)):
        state[i] = ror(state[i], (rnd+i) & 7)
    # Undo byte mask (XOR is self-inverse).
    for i in range(len(state)):
        state[i] ^= (rnd*41 + i*17) & 255
print(state.decode())
```

Why right-to-left for chained XOR? Encryption overwrote `state[i]` using the already changed `state[i-1]`. Reverse the chain from the final index downward to preserve the values needed for inversion.

Adapt: write the operations of one round in forward order, then reverse that list. For left-rotate-by-n, use right-rotate-by-n. XOR masks can be applied again. For addition modulo 256, subtract the added value. For a byte permutation, use the inverse permutation. Do not claim every hash is reversible: this works only because of this particular design.

10 SecureBank 1 – Salsa20 bit-flipping

Clue: Salsa20 encrypts a JSON transaction, outputs `nonce + ciphertext`, and there is no authentication tag. Salsa20 is a stream cipher, so $C = P \oplus K$. To change only a known byte from p to p', modify its ciphertext byte with the mask $p \oplus p'$.

```
import socket
HOST, PORT = 'cns-test.idlab.uantwerpen.be', 50001
with socket.socket() as s:
    s.connect((HOST, PORT))
    incoming = b''
    while b'Submit the transaction (hex):' not in incoming:
        chunk = s.recv(4096)
        if not chunk: raise RuntimeError('Connection closed')
        incoming += chunk
    text = incoming.decode()
    blob = text.split('Your pending transaction:\n', 1)[1].split()[0]
    raw = bytes.fromhex(blob)
    nonce, ciphertext = raw[:8], raw[8:]
    known = b'{"from":"alice","to":"bob","amount":"010.00"}'
    assert len(known) == len(ciphertext)
    pos = known.index(b'010.00') + 1  # index of '1'
    changed = bytearray(ciphertext)
    changed[pos] ^= ord('1') ^ ord('0')
    payload = (nonce + changed).hex().encode() + b'\n'
    s.sendall(payload)
    print(s.recv(4096).decode(errors='replace'))
```

Code explanation: `recv()` can split text at any point, so read until the prompt. `bytes.fromhex` converts the received hex to bytes. The *given* function used `secrets.token_bytes(8)` and returned `nonce + ciphertext`; therefore `raw[:8]` contains 8 *bytes*, not bits. The `index` finds the exact byte to change, while `bytearray` allows editing. The original nonce is kept to preserve the same keystream.

Adapt: do not guess the nonce size or position; read the protocol. Replace the known JSON and the old/new characters. If the desired text has the same byte length, use a whole-span mask: `changed[pos+i] ^= old[i] ^ new[i]`. If you do not know the original byte, you cannot predict the resulting byte by this technique. This attack changes content, not the secret key. **Defence:** authenticated encryption, for example ChaCha20-Poly1305.

11 SecureBank 2 – truncated SHA-256 collision

Clue: the bank stores only the first 8 hex characters of SHA-256, so the fingerprint is 32 bits. By the birthday effect, searching for *any two* inputs with the same 32-bit result takes roughly 2^{16} attempts in order of magnitude. That differs from a fixed-target second-preimage search (about 2^{32}). The server accepts payments with `reference, from_account, to_account, amount, currency, timestamp`.

```
import hashlib, json, socket
from datetime import datetime, timezone

def canonicalize(p):
    return json.dumps(p, sort_keys=True, separators=(',', ':'),
                      ensure_ascii=True).encode()

def find_pair(timestamp, limit=300000):
    base = dict(reference='', from_account='SIM-ALICE',
```

```
                to_account='SIM-BOB', amount=10,
                currency='EUR', timestamp=timestamp)
    seen = {}
    for i in range(limit):
        base['reference'] = f'PAYMENT-{i}'
        fingerprint = hashlib.sha256(canonicalize(base)).hexdigest()[:8]
        if fingerprint in seen:
            p1, p2 = base.copy(), base.copy()
            p1['reference'] = seen[fingerprint]
            return p1, p2, fingerprint
        seen[fingerprint] = base['reference']
    raise RuntimeError('Try a larger search range or new timestamp')

stamp = datetime.now(timezone.utc).isoformat()
p1, p2, fp = find_pair(stamp)
print(fp, p1['reference'], p2['reference'])
with socket.create_connection(('cns-test.idlab.uantwerpen.be', 50003), 5) as s:
    s.settimeout(5)
    s.sendall(canonicalize(p1) + b'\n' + canonicalize(p2) + b'\n')
    print(s.recv(8192).decode(errors='replace'))
```

Code explanation: `sort_keys=True` sorts JSON fields; `separators` removes spaces. `hexdigest()[:8]` keeps 8 hex characters, not 8 bytes. The dictionary `seen` maps each fingerprint to its first reference; the second occurrence forms a collision. `payment.copy()` keeps two separate records. The timestamp must be recent; the exercise server required the previous minute.

Confirmed example: `PAYMENT-1019` and `PAYMENT-163643` produced `0abbe7db`, and the server returned `cns{legacy_recor` The reference numbers depend on the timestamp and must not be copied as a universal solution. **Adapt:** change the canonical format, hash algorithm, truncation size, required fields, or time range according to the new statement. Do not regenerate the timestamp between finding and sending your pair.

12 Mars Relay – hash chain prediction

Clue: current code is 64 hex characters (32 bytes), and the server asks for the next code. The confirmed rule was $C_{n+1} = \text{SHA256}(\text{bytes.fromhex}(C_n))$. Hashing the 64 ASCII hex characters gives a *different* result.

```
import socket, re, hashlib
HOST, PORT = 'cns-test.idlab.uantwerpen.be', 50006
with socket.create_connection((HOST, PORT), 5) as s:
    s.settimeout(5)
    buf = b''
    while not re.search(rb'CURRENT CODE:\s*[0-9a-fA-F]{64}', buf):
        part = s.recv(4096)
        if not part: raise RuntimeError('Missing current code')
        buf += part
    current = re.search(rb'CURRENT CODE:\s*([0-9a-fA-F]{64})', buf).group(1)
    answer = hashlib.sha256(bytes.fromhex(current.decode())).hexdigest()
    s.sendall(answer.encode() + b'\n')
    print(s.recv(4096).decode(errors='replace'))
```

Confirmed result: `cns{hash_chains_are_not_totp}`. **Adapt:** do not assume the next value is always SHA-256. Test whether the hash input is raw bytes, text, a counter, timestamp or secret. A real TOTP uses a secret key and time counter; merely hashing a public previous token makes future codes predictable.

13 Signed, Sealed, Misinterpreted – HMAC and duplicate JSON keys

Clue: `/challenge` supplies signed JSON and `/vault` checks an `X-Signature` header. Original `{"action":"view","role":"us` gives `Access denied`; changing role to admin directly gives `Invalid signature`. The confirmed bypass sent `{"action":"view","role":"admin","role":"user"}` using the legitimate signature.

```
import requests
BASE = 'http://cns-test.idlab.uantwerpen.be:50007'
ch = requests.get(BASE + '/challenge', timeout=10).json()
headers = {'X-Signature': ch['signature'],
           'Content-Type': 'application/json'}
raw_json = '{"action":"view","role":"admin","role":"user"}'
r = requests.post(BASE + '/vault', headers=headers,
                  data=raw_json.encode(), timeout=10)
print(r.status_code, r.text)
```

Code explanation: use `data=` to preserve duplicate field names as raw JSON. With `json=`, Python would serialize a dictionary, which cannot contain two separate values for the same key. The outcome shows a mismatch

between signature verification and access control. We cannot see the internal code, but the evidence fits a verifier using the last role (user) and an authorizer using the first role (admin).

Confirmed flag: `cns{canonicalization_is_critical}`. **Adapt:** inspect whether the server signs raw bytes, re-serialized JSON, selected fields, or a canonical form; test duplicate keys, Unicode escapes, order and extra fields separately. A valid HMAC alone does not grant permission. A robust server rejects duplicate keys and verifies and authorizes the same parsed object.

14 Vault JWT – unsigned `alg: none` token

Clue: login returns a JWT: three base64url parts separated by dots, `header.payload.signature`. The example guest payload was `{"user":"guest","role":"user"}`, with header `{"alg":"HS256","typ":"JWT"}`. The laboratory server incorrectly accepted a new token with `{"alg":"none"}` and admin role, with an empty signature.

```
import requests, json, base64
BASE = 'http://cns-test.idlab.uantwerpen.be:50009'

def b64_json(obj):
    data = json.dumps(obj, separators=(',', ':')).encode()
    return base64.urlsafe_b64encode(data).decode().rstrip('=')

original = requests.get(BASE + '/login', timeout=10).json()['token']
head, pay, sig = original.split('.')
for piece in (head, pay):
    raw = base64.urlsafe_b64decode(piece + '=' * (-len(piece) % 4))
    print(json.loads(raw))

unsigned = (b64_json({'alg': 'none', 'typ': 'JWT'}) + '.' +
            b64_json({'user': 'guest', 'role': 'admin'}) + '.')
r = requests.get(BASE + '/admin',
                 headers={'Authorization': 'Bearer ' + unsigned},
                 timeout=10)
print(r.status_code, r.text)
```

Confirmed flag: `CNS{never_trust_the_alg_header}`. **Code explanation:** a JWT is encoded, not encrypted. `rstrip('=')` removes Base64URL padding. `alg: none` means no signature, so the third part is empty and the token ends in a dot. The bug is server-side trust in an attacker-controlled algorithm setting. **Adapt:** the role field might be `is_admin` or `permissions`, and the endpoint or header format might differ. Decode first, then match exact names; do not assume every JWT library accepts unsigned tokens.

15 Updated practical solutions: use these versions first

This section contains the updated methods we tested after making the first guide. For the exam, start here for AES2, Better Hash, Enigma and Replanting. The earlier chapters give background and simpler alternatives. **Only use files supplied in the new question.** Never use an earlier decrypted result as a known plaintext unless the question gives it.

15.1 AES2: discover ECB alignment, infer BMP masks, solve MITM

Recognise it: two short AES keys; BMP input; repeated 16-byte ciphertext blocks. **Why it works:** ECB preserves equal-block patterns, BMP may contain predictable colour masks, and MITM finds the same middle block from two directions. In this specific lab, encryption starts at byte 54, even though pixels start at 138. The standard mask values are a *hypothesis*, not guaranteed by every BMP.

```
from collections import Counter
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
import struct

raw = open('enc.bmp', 'rb').read()
assert raw[:2] == b'BM'
file_size = struct.unpack_from('<I', raw, 2)[0]
pixel_offset = struct.unpack_from('<I', raw, 10)[0]
dib_size = struct.unpack_from('<I', raw, 14)[0]
bits = struct.unpack_from('<H', raw, 28)[0]
compression = struct.unpack_from('<I', raw, 30)[0]
print('Header:', file_size, pixel_offset, dib_size, bits, compression)

OFFSET = 54    # Found by checking alignment; NOT always 54
body = raw[OFFSET:]
assert len(body) % 16 == 0
```

```
blocks = [body[i:i+16] for i in range(0, len(body), 16)]
print('Most common blocks:', [(x.hex(), n)
      for x,n in Counter(blocks).most_common(4)])

# In this specific 32-bit BI_BITFIELDS BMP, the first
# encrypted block contains four 32-bit colour masks.
assert dib_size >= 56 and bits == 32 and compression == 3
candidates = {
    'BGRA': struct.pack('<4I', 0x00FF0000, 0x0000FF00,
                                0x000000FF, 0xFF000000),
    'RGBA': struct.pack('<4I', 0x000000FF, 0x0000FF00,
                                0x00FF0000, 0xFF000000),
}
C = raw[OFFSET:OFFSET+16]
KEY_BITS = 20

def make_key(k):
    return k.to_bytes(3, 'big') + b'\x00'*13

def search(P, C):
    table = {}
    for k1 in range(1 << KEY_BITS):
        if k1 % 100000 == 0: print('K1', k1, flush=True)
        mid = AES.new(make_key(k1), AES.MODE_ECB).encrypt(P)
        table[mid] = k1
    for k2 in range(1 << KEY_BITS):
        if k2 % 100000 == 0: print('K2', k2, flush=True)
        mid = AES.new(make_key(k2), AES.MODE_ECB).decrypt(C)
        if mid in table:
            yield table[mid], k2

def decrypt_body(k1, k2):
    c1 = AES.new(make_key(k1), AES.MODE_ECB)
    c2 = AES.new(make_key(k2), AES.MODE_ECB)
    return unpad(c1.decrypt(c2.decrypt(body)), 16)

keys = None
for name, P in candidates.items():
    print('Trying', name, flush=True)
    for k1, k2 in search(P, C):
        try:
            plain_body = decrypt_body(k1, k2)
        except ValueError:
            continue
        restored = raw[:OFFSET] + plain_body
        if len(restored) == file_size and plain_body[:16] == P:
            keys = k1, k2
            open('recovered.bmp', 'wb').write(restored)
            break
    if keys: break
assert keys, 'Try a different P/C pair, offset or mode'
print('Keys:', keys)

promo_hex = ('df7c7dfbd5e1ba2af108d658132d5f20'
 'd34de27da16375ceabd7a79cbfe73824'
 '4027a4a0b26ec498a2c223a545b9567c'
 '20bf423bb38514e6cd6f043ab9e23a84'
 'ffa6745294c224068e1cffd9e8c9e294')
k1, k2 = keys
c1 = AES.new(make_key(k1), AES.MODE_ECB)
c2 = AES.new(make_key(k2), AES.MODE_ECB)
print(unpad(c1.decrypt(c2.decrypt(bytes.fromhex(promo_hex))),16).decode())
```

Change if the question changes: the file path; encryption offset; plaintext candidates; `KEY_BITS`; key encoding in `make_key`; encryption mode and padding. Do not blindly reuse BMP masks in a different file format. **If it fails:** print offsets 0–15 relative to your suspected start, compare repeated blocks, inspect BMP header, and test another *justified* known plaintext block. **Check:** correct padding, expected original file length, readable reconstructed image, and meaningful decrypted promo code. The recovered keys in our particular lab were 269642 and 942733, but do not hardcode them in a new task.

15.2 Better Hash: invert every step in reverse order

Recognise it: custom so-called hash made of XOR, ROL, chained XOR and byte-array rotation. If all operations are reversible, the output is not a secure one-way hash.

```
HASH = ('5f070b54361613bca31a123e95321cff154ba64f'
```

```
        '549bb34fdc7688f10efb569aed4ccbd6ba')
ROUNDS, R_MULT, I_MULT = 8, 41, 17

def ror(x, n):
    n %= 8
    return ((x >> n) | (x << (8-n))) & 255

def reverse_hash(hash_hex):
    state = bytearray.fromhex(hash_hex)
    for rnd in reversed(range(ROUNDS)):
        state = state[-1:] + state[:-1]  # undo left array move
        for i in range(len(state)-1, 0, -1):
            state[i] ^= state[i-1]       # undo XOR chain
        for i in range(len(state)):
            state[i] = ror(state[i], (rnd+i) & 7)
            state[i] ^= (rnd*R_MULT + i*I_MULT) & 255
    return bytes(state)

print(reverse_hash(HASH).decode())
```

Change: ROUNDS, multipliers, bit rotation rule, or order of operations. **If the order changes**, reverse the new order, not just the values. **Check:** run the original hash function on the recovered plaintext and compare exact hex output.

15.3 Enigma: stepping and inverse permutations

Recognise it: one or more rotor alphabets, starting rotations, and an exact stepping rule. The important detail is that decryption uses the *same rotor motion* but applies inverse substitutions in reverse order.

```
ABC = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
ROTORS = ['MACHINEBDFGJKLOPQRSTUVWXYZ',
          'ORBITACDEFGHJKLMNPQSUVWXYZ',
          'CIPHERABDFGJKLMNOQSTUVWXYZ']
START = [0, 0, 0]
CIPHERTEXT = 'HKV{BSBU-B-SJYZNNB-DNFW-IXH-YTA-RPL-RTXOLWFQ}'

def rotate(r, n):
    n %= 26
    return r[n:] + r[:n]

def transform(text, decrypt=False):
    rotors = [rotate(r, n) for r,n in zip(ROTORS, START)]
    count, out = 0, ''
    for ch in text.upper():
        if ch not in ABC:
            out += ch  # punctuation does not advance the rotors
            continue
        count += 1
        rotors[0] = rotate(rotors[0], 1)  # before this letter
        if count % 26 == 0: rotors[1] = rotate(rotors[1], 1)
        if count % 676 == 0: rotors[2] = rotate(rotors[2], 1)
        index = ABC.index(ch)
        if decrypt:
            for r in reversed(rotors):
                index = r.index(ABC[index])
        else:
            for r in rotors:
                index = ABC.index(r[index])
        out += ABC[index]
    return out

for r in ROTORS: assert len(r)==26 and set(r)==set(ABC)
plain = transform(CIPHERTEXT, decrypt=True)
print(plain)
assert transform(plain) == CIPHERTEXT
```

Change: rotor strings, starting rotations, stepping timing, whether punctuation steps, rotor order. **Check:** re-encrypt the result using the same starting positions, and require the original ciphertext.

15.4 Replanting: stream XOR + flexible LCG state search

Recognise it: known public plaintext with its ciphertext, plus a secret ciphertext; source code shows `state = (A*state+C)%M` and shifts out 8 bits. First recover stream bytes with XOR, then reconstruct the hidden state. The LCG attack is needed if the known public plaintext is shorter than the secret.

```python
from pathlib import Path
A, C, M = 1103515245, 12345, 2**31
SHIFT, STATE_BITS, OUT_BITS = 16, 31, 8
known_pt = b'STATUS: SERVER ONLINE'
known_ct = bytes.fromhex(Path('public.txt').read_text().strip())
secret_ct = bytes.fromhex(Path('secret.txt').read_text().strip())
assert len(known_pt) == len(known_ct)
known_k = bytes(p^c for p,c in zip(known_pt, known_ct))

def step(s): return (A*s+C) % M
def out(s): return (s >> SHIFT) & ((1<<OUT_BITS)-1)
def stream(seed, n):
    s = seed
    result = bytearray()
    for _ in range(n):
        s = step(s)
        result.append(out(s))
    return bytes(result)

assert M == 2**STATE_BITS and OUT_BITS == 8
high_bits = STATE_BITS - SHIFT - OUT_BITS
first_state = None
for hi in range(1<<high_bits):
    for lo in range(1<<SHIFT):
        s0 = (hi << (SHIFT+OUT_BITS)) | (known_k[0]<<SHIFT) | lo
        s = s0
        for expected in known_k[1:]:
            s = step(s)
            if out(s) != expected: break
        else:
            first_state = s0
            break
    if first_state is not None: break
assert first_state is not None
seed = ((first_state-C) * pow(A,-1,M)) % M
assert stream(seed, len(known_k)) == known_k
plain = bytes(c^k for c,k in zip(secret_ct, stream(seed,len(secret_ct))))
print(plain.decode())
```

Change: `A`, `C`, `M`, `SHIFT`, output bits, ciphertext files, known plaintext. The code assumes a power-of-two modulus, 8-bit outputs, and that both messages start from the same seed. **If too many states:** this brute-force method may be too slow; consider using modular constraints. **Check:** replay the complete known keystream before decrypting the secret.

15.5 Quick practical fallback: when you cannot complete a challenge

For a partly solved exercise, write a short method in plain English, showing the key equation and what you tested. For example: "I found repeated 16-byte blocks, so ECB is likely. I would get a known plaintext-ciphertext block from a predictable file field, use MITM to find both 20-bit keys, and test the keys by decrypting another block." Or: "I know a public plaintext and ciphertext, so I can XOR them to recover keystream bytes. If the generator is a weak LCG, I can test the unknown state bits and regenerate the rest." This describes a real procedure rather than giving unrelated theory.

16 Universal exam workflow and error checklist

16.1 How to solve a changed question without AI

1. **Identify input types:** printed hex, raw bytes, JSON, base64url, uppercase alphabet or files. Write down known plaintext, ciphertext, key hints and nonce/IV layout.
2. **Read the encryption formula:** stream XOR? repeated-key substitution? block cipher mode? custom reversible rounds? hash or MAC? Do not guess from a challenge title alone.
3. **Find what is weak:** key space too small, repeated nonce, predictable chain, short fingerprint, duplicate JSON parser, missing signature verification.
4. **Copy a matching template:** edit only input data and the parameters justified by the task. Label assumptions clearly.
5. **Validate locally:** check lengths, padding, decoding, reverse encryption or matching hash values; do not treat readable text as mathematical proof alone.

6. **Communicate with server:** identify correct endpoint/method; read TCP prompts completely; send the exact expected encoding and newline; inspect status codes.

7. **Record evidence:** ciphertext, selected parameters, test outcome, flag. If a test fails, report what was ruled out and change one assumption at a time.

16.2 Common Python problems

- `bytes.fromhex()` fails if length is odd or includes nonhex characters. Do not guess a missing digit.
- `zip(a,b)` silently stops at the shortest input. Always assert matching lengths for whole-message XOR.
- `^` is XOR in Python, *not* power. Exponentiation is `**`. `ord()` gives character code.
- `%` means remainder/modulo. `range(2**20)` is about one million keys.
- `recv(4096)` may return part of a TCP message, not a complete logical line. Read until a delimiter or end.
- For HTTP, `requests.post(...,json=obj)` serializes an object; `data=raw_json` preserves exact bytes.
- Use `datetime.now(timezone.utc)` if server timestamps must be recent.
- A valid hash collision must use two *different* original messages and identical truncated fingerprints.
- A valid HMAC cannot be forged by simply changing its signed input; find a parser, encoding or logic flaw instead.
- The new question may change offsets, key lengths, block mode, endianness, rotation direction and alphabet. Always re-check those details.

Exam warning

The Caesar shift and the working CTF flags were confirmed from the supplied inputs and prior successful outputs. The Belgian word-list cipher and the Dutch corrupted hex string cannot be marked as solved without the exact attached word list / corrected ciphertext and a verified decoding. In AES2, test known-block guesses and key pairs instead of treating the first potential match as proof.